QUALITY ASSURANCE & VERIFICATION

Testing Plan

Project: CloudGuard IoT — Unified Autonomous Cloud & IoT Security Platform | **Status:** Proposed

1. Introduction

This document defines the testing strategy for *CloudGuard IoT — Unified Autonomous Cloud & IoT Security Platform*.

The purpose of testing is to verify that the platform's main features work correctly, security controls are enforced, data is handled consistently, and the dashboard provides reliable information.

The project supports simulated IoT events for academic demonstrations. AWS testing and response actions must use authorized test environments and allowlisted resources.

> **Note:** This is a proposed testing plan. Test cases must not be marked as passed until they have actually been executed.

2. Testing Objectives

- Verify the main functional requirements.
- Validate API requests and responses.
- Verify database operations and relationships.
- Confirm that only approved sources can submit security events.
- Verify risk-assessment and incident workflows.
- Ensure unauthorized users cannot perform restricted operations.
- Prevent response actions against non-allowlisted resources.
- Check dashboard usability and data consistency.
- Identify and document defects before the final demonstration.

3. Testing Scope

3.1 In Scope

- Authentication and role-based authorization.
- Dashboard statistics and recent activity.
- Device management.
- Cloud-resource management.
- Security-event ingestion and validation.
- Risk assessment.
- Incident management.
- Response-action requests and approvals.

- Audit-log creation and retrieval.
- Database integrity.
- API error handling.
- UI and usability testing.
- Simulated IoT event generation and processing.

3.2 Out of Scope

Unless separately implemented and authorized:

- Testing third-party or production infrastructure without permission.
- Attacking external systems.
- Executing disruptive actions on production cloud resources.
- Formal compliance certification.
- Guaranteeing production-scale performance without a dedicated test environment.

4. Testing Types

Testing Type	Purpose
Functional Testing	Verify that features behave according to requirements
API Testing	Verify endpoints, payloads, status codes, and error handling
Integration Testing	Verify communication between modules, database, AWS integrations, and simulator
Database Testing	Verify data persistence and relationships
Security Testing	Verify authentication, authorization, and security restrictions
UI Testing	Verify navigation, forms, and dashboard layout
Usability Testing	Check that users can complete common workflows
Regression Testing	Ensure updates do not break existing functionality
Performance Testing	Evaluate response time under an agreed workload, if supported

5. Test Environment

The proposed testing environment includes:

Component	Proposed Environment
Frontend	Next.js / React
Backend	Next.js Route Handlers or selected API layer
Database	PostgreSQL test database

Component	Proposed Environment
ORM	Drizzle ORM
Cloud Integration	AWS test account and authorized test resources, if available
IoT Source	Virtual IoT event simulator
Browser	Chrome, Firefox, or Edge
Source Control	Git and GitHub

Use separate test credentials and configuration. Do not use production secrets or real customer data.

6. Entry and Exit Criteria

6.1 Entry Criteria

Testing can begin when:

- The feature under test is available.
- Database schema and test data are prepared.
- Test users and roles are configured.
- Required API endpoints or UI workflows are accessible.
- Dependencies and test configuration are documented.

6.2 Exit Criteria

A testing cycle may be considered complete when:

- Planned high-priority test cases have been executed.
- Critical and high-severity defects are fixed or documented.
- Failed tests have been investigated and retested.
- Security restrictions for privileged operations have been verified.
- Results and known limitations are documented.

7. Functional Test Cases

The following cases are proposed and should be executed during testing.

ID	Feature	Test Scenario	Expected Result	Priority
TC-001	Authentication	Sign in with valid credentials	User is authenticated and reaches dashboard	High
TC-002	Authentication	Sign in with invalid credentials	Access is denied with a safe error message	High
TC-003	Authorization	Viewer attempts administrator-only operation	Request is rejected	High

ID	Feature	Test Scenario	Expected Result	Priority
TC-004	Dashboard	Open dashboard with test data	Statistics and recent activity appear correctly	High
TC-005	Devices	Register device with valid fields	Device is stored and appears in the list	High
TC-006	Devices	Submit device without required name	Validation error; no invalid record is created	Medium
TC-007	Devices	Retrieve unknown device ID	Not-found response is returned	Medium
TC-008	Cloud Resources	Register authorized test resource	Resource is stored and displayed	High
TC-009	Cloud Resources	Submit invalid resource fields	Validation error is returned	Medium
TC-010	Event Ingestion	Submit valid simulated IoT event	Event is validated and stored	High
TC-011	Event Ingestion	Submit invalid severity value	Request is rejected	High
TC-012	Event Ingestion	Submit event from unapproved source	Request is rejected or quarantined according to policy	High
TC-013	Risk Assessment	Process an event eligible for assessment	Risk score is generated within 0–100	High
TC-014	Incident Management	Open existing incident	Details and related events are displayed	High
TC-015	Incident Management	Update incident with allowed status	Update is saved and displayed	High
TC-016	Incident Management	Submit invalid incident status	Update is rejected; previous status remains unchanged	Medium
TC-017	Response Actions	Request action for allowlisted test target	Request is created and awaits approval	High
TC-018	Response Actions	Request action for non-allowlisted target	Request is rejected and no action is executed	Critical
TC-019	Response Actions	Attempt approval without permission	Approval is denied	Critical
TC-020	Audit Logs	Perform auditable administrative operation	Audit record includes actor, action, target, and timestamp	High
TC-021	Audit Logs	Access audit logs without permission	Access is denied	High

ID	Feature	Test Scenario	Expected Result	Priority
TC-022	Error Handling	Request nonexistent resource	Consistent error response without internal details	Medium

8. API Testing Scenarios

For each implemented endpoint, test the following where applicable:

1. Valid request with all required fields.
2. Missing required fields.
3. Incorrect data types.
4. Invalid enum values.
5. Malformed JSON.
6. Missing authentication.
7. Insufficient user permissions.
8. Unknown resource identifier.
9. Duplicate submission.
10. Unexpected server or dependency error.

Verify that:

- HTTP status codes match the API documentation.
- Response bodies follow the documented structure.
- Sensitive information is not returned.
- Invalid requests do not create partial or inconsistent records.

9. Integration Testing

9.1 IoT Simulator to Event Ingestion

Steps:

1. Generate a simulated IoT event.
2. Submit it to the event-ingestion API.
3. Verify that it is validated and stored.
4. Confirm that it appears in the event list or dashboard.

Expected Result: The event is processed through the approved ingestion workflow and becomes available for investigation.

9.2 AWS Event Source to Event Ingestion

If an AWS integration has been implemented:

1. Generate an approved event in the AWS test environment.
2. Verify that the integration receives or forwards the event.
3. Confirm that the application validates the source and payload.
4. Verify that the event is stored and displayed.

Expected Result: Authorized AWS events are processed according to the configured integration.

If the integration is not implemented, use a clearly labeled simulated event. Do not mark the AWS integration test as passed.

9.3 Event to Risk Assessment and Incident

Steps:

1. Submit a valid test event.
2. Run the risk-assessment process.
3. Verify that a score and explanation are generated.
4. Check whether the event is linked to an existing incident or creates a new incident according to the correlation rules.

Expected Result: Event, assessment, and incident records remain correctly related.

9.4 Incident to Response Action and Audit Log

Steps:

1. Select a test incident.
2. Submit a response request for an allowlisted test target.
3. Verify that approval is required.
4. Approve or reject using an authorized account.
5. Verify the action status and audit records.

Expected Result: Authorization and approval are enforced, and important workflow transitions are recorded. Execution may be simulated if a real integration is unavailable.

10. Security Testing

ID	Security Check	Expected Result
ST-001	Access protected endpoint without authentication	Request is rejected
ST-002	Viewer attempts administrator-only operation	Request is rejected
ST-003	Submit malformed or unexpected input	Input is handled safely
ST-004	Change resource ID to access unauthorized resource	Access is denied

ID	Security Check	Expected Result
ST-005	Target non-allowlisted resource with response action	Request is rejected; no action executes
ST-006	Approve action using unauthorized account	Approval is rejected
ST-007	Inspect API errors for exposed secrets or stack traces	Sensitive details are not exposed
ST-008	Review application and audit logs	Passwords and tokens are not recorded

All security tests must be performed only on systems and resources for which the project team has authorization.

11. Database Testing

Verify that:

- Every entity has a valid primary key.
- Foreign keys reference existing records.
- Required fields cannot be stored as null.
- Unique identifiers remain unique.
- Update and delete operations follow intended relationship rules.
- Events remain associated with the correct source.
- Risk assessments reference the correct event or incident context.
- Incident-event relationships are preserved.
- Response actions reference the intended incident and target.
- Audit records preserve actor and action details.
- Failed transactions do not leave inconsistent partial data.

Core Database Entities

- Device
- CloudResource
- SecurityEvent
- RiskAssessment
- Incident
- IncidentEvent
- ResponseAction
- AuditLog

12. UI and Usability Testing

Verify that:

- Main navigation links open the correct pages.
- Dashboard statistics match available test data.
- Tables handle empty, loading, and error states.
- Forms show clear validation messages.
- Incident details display related events and risk information.
- Privileged actions are clearly distinguished and require confirmation where appropriate.
- The layout remains usable on desktop and smaller screens.
- Arabic and English text, if supported, display correctly.
- Severity and status are not communicated by color alone.

13. Defect Management

Each defect should be recorded using the following information:

Field	Description
Defect ID	Unique reference
Title	Short description
Environment	Where the defect occurred
Steps to Reproduce	Exact reproduction steps
Expected Result	Intended behavior
Actual Result	Observed behavior
Severity	Critical, High, Medium, or Low
Priority	Order for fixing
Status	New, In Progress, Fixed, Retest, or Closed
Evidence	Screenshot or sanitized logs/request/response

Severity Definitions

- **Critical:** Security boundary bypass or unauthorized dangerous response action.
- **High:** Core workflow unavailable or materially incorrect.
- **Medium:** Defect with a reasonable workaround.
- **Low:** Minor visual, wording, or usability issue.

14. Test Execution Record

Update this table only after tests have actually been executed.

Test Case ID	Execution Date	Tester	Actual Result	Status	Notes / Evidence
TC-001	Not executed	—	—	Not Run	Update after execution
TC-002	Not executed	—	—	Not Run	Update after execution
TC-003	Not executed	—	—	Not Run	Update after execution
TC-010	Not executed	—	—	Not Run	Update after execution
TC-013	Not executed	—	—	Not Run	Update after execution
TC-018	Not executed	—	—	Not Run	Update after execution
TC-020	Not executed	—	—	Not Run	Update after execution

Suggested status values: Not Run, Pass, Fail, Blocked.

Never mark a test as passed without executing it and checking the observed result.

15. Risks and Limitations

- AWS event sources may be simulated if integration access is unavailable.
- Real response actions may be replaced by safe simulations for the academic demonstration.
- Results depend on implemented features and the test environment.
- Performance results from a small academic environment do not establish production-scale capacity.
- Unimplemented endpoints and integrations must be marked as not tested, not passed.

16. Testing Deliverables

The testing phase should produce:

- Completed test execution records.
- Defect log with severity and status.
- Sanitized API request and response evidence.
- Screenshots of key application workflows.
- Integration results for implemented integrations.
- Summary of passed, failed, blocked, and not-run tests.
- Known limitations and recommended future improvements.